Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo, Norway[1]

May 14, 2025

Plan for the week of May 12-16

1. Quantum Boltzmann Machines: Theory and Implementation
   - Quantum neural networks, wrapping up discussions from last week (see notes from last week at https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/pub/week15/ipynb/week15.ipynb)
   - Classical Boltzmann Machines (BMs)
   - Restricted Quantum Boltzmann Machines (RQBM)
   - Training Quantum Boltzmann Machines
   - Practical Implementation with PennyLane
2. Summary of course and work on project 2

Introduction

Quantum Boltzmann Machines (QBM extend the classical Boltzmann machine (a probabilistic neural network) into the quantum domain. QBMs promise richer representations by leveraging quantum superposition and entanglement, potentially capturing correlations that classical models cannot .

In these notes, we review first classical Boltzmann machines and restricted Boltzmann machines (RBMs). Thereafter we introduce QBMs and their restricted variant (RQBM), discuss training methods, and illustrate practical implementation using PennyLane.

Classical Boltzmann machines (aka Energy models)

We define a domain $\boldsymbol{X}$ of stochastic variables $\boldsymbol{X} = \{x_0, x_1, \ldots, x_{n-1}\}$ with a pertinent probability distribution

$$p(\boldsymbol{X}) = \prod_{x_i \in \boldsymbol{X}} p(x_i),$$

where we have assumed that the random varaibles x_i are all independent and identically distributed (iid).
We will now assume that we can defined this function in terms of optimization parameters $\boldsymbol{\Theta}$, which could be the biases and weights of deep network, and a set of hidden variables we also assume to be random variables which also are iid. The domain of these variables is $\boldsymbol{H} = \{h_0, h_1, \ldots, h_{m-1}\}$.

Probability model

We define a probability

$$p(x_i, h_j; \boldsymbol{\Theta}) = \frac{f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

where $f(x_i, h_j; \boldsymbol{\Theta})$ is a function which we assume is larger or equal than zero and obeys all properties required for a probability distribution and $Z(\boldsymbol{\Theta})$ is a normalization constant. Inspired by statistical mechanics, we call it often for the partition function. It is defined as (assuming that we have discrete probability distributions)

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}).$$

Marginal and conditional probabilities

We can in turn define the marginal probabilities

$$p(x_i; \boldsymbol{\Theta}) = \frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

and

$$p(h_i; \boldsymbol{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i, h_j; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})}.$$

Change of notation

Note the change to a vector notation. A variable like $\boldsymbol{x}$ represents now a specific **configuration**. We can generate an infinity of such configurations. The final partition function is then the sum over all such possible configurations, that is

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

changes to

$$Z(\boldsymbol{\Theta}) = \sum_{\boldsymbol{x}} \sum_{\boldsymbol{h}} f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}).$$

If we have a binary set of variable x_i and h_j and M values of x_i and N values of h_j we have in total 2^M and 2^N possible $\boldsymbol{x}$ and $\boldsymbol{h}$ configurations, respectively.
We see that even for the modest binary case, we can easily approach a number of configuration which is not possible to deal with.

Optimization problem

At the end, we are not interested in the probabilities of the hidden variables. The probability we thus want to optimize is

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} p(x_i;\boldsymbol{\Theta}) = \prod_{x_i \in \boldsymbol{X}} \left(\frac{\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j;\boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})} \right),$$

which we rewrite as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} \left(\sum_{h_j \in \boldsymbol{H}} f(x_i, h_j;\boldsymbol{\Theta}) \right).$$

Further simplifications

We simplify further by rewriting it as

$$p(\boldsymbol{X};\boldsymbol{\Theta}) = \frac{1}{Z(\boldsymbol{\Theta})} \prod_{x_i \in \boldsymbol{X}} f(x_i;\boldsymbol{\Theta}),$$

where we used $p(x_i;\boldsymbol{\Theta}) = \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j;\boldsymbol{\Theta})$. The optimization problem is then

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} p(\boldsymbol{X};\boldsymbol{\Theta}).$$

Optimizing the logarithm instead

Computing the derivatives with respect to the parameters $\boldsymbol{\Theta}$ is easier (and equivalent) with taking the logarithm of the probability. We will thus optimize

$$\arg \max_{\boldsymbol{\Theta} \in \mathbb{R}^p} \log p(\boldsymbol{X}; \boldsymbol{\Theta}),$$

which leads to

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = 0.$$

Expression for the gradients

This leads to the following equation

$$\nabla_{\boldsymbol{\Theta}} \log p(\boldsymbol{X}; \boldsymbol{\Theta}) = \nabla_{\boldsymbol{\Theta}} \left(\sum_{x_i \in \boldsymbol{X}} \log f(x_i; \boldsymbol{\Theta}) \right) - \nabla_{\boldsymbol{\Theta}} \log Z(\boldsymbol{\Theta}) = 0.$$

The first term is called the positive phase and we assume that we have a model for the function f from which we can sample values. The second term is called the negative phase and is the one which leads to more difficulties.

The derivative of the partition function

The partition function, defined above as

$$Z(\boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} \sum_{h_j \in \boldsymbol{H}} f(x_i, h_j; \boldsymbol{\Theta}),$$

is in general the most problematic term. In principle both x and h can span large degrees of freedom, if not even infinitely many ones, and computing the partition function itself is often not desirable or even feasible. The above derivative of the partition function can however be written in terms of an expectation value which is in turn evaluated using Monte Carlo sampling and the theory of Markov chains, popularly shortened to MCMC (or just MC^2).

Final expression

Summarizing, we have

$$\nabla_{\mathbf{\Theta}} \log Z(\mathbf{\Theta}) = \frac{\sum_{x_i \in \boldsymbol{X}} f(x_i; \mathbf{\Theta}) \nabla_{\mathbf{\Theta}} \log f(x_i; \mathbf{\Theta})}{Z(\mathbf{\Theta})},$$

which is the expectation value of $\log f$

$$\nabla_{\mathbf{\Theta}} \log Z(\mathbf{\Theta}) = \sum_{x_i \in \boldsymbol{X}} p(x_i; \mathbf{\Theta}) \nabla_{\mathbf{\Theta}} \log f(x_i; \mathbf{\Theta}),$$

that is

$$\nabla_{\mathbf{\Theta}} \log Z(\mathbf{\Theta}) = \mathbb{E}(\log f(x_i; \mathbf{\Theta})).$$

This quantity is evaluated using Monte Carlo sampling, with Gibbs sampling as the standard sampling rule.

Kullback-Leibler divergence

The Kullback–Leibler (KL) divergence, labeled D_{KL}, measures how one probability distribution p diverges from a second expected probability distribution q, that is

$$D_{KL}(p\|q) = \int_x p(x) \log \frac{p(x)}{q(x)} dx.$$

The KL-divergence D_{KL} achieves the minimum zero when $p(x) == q(x)$ everywhere.
Note that the KL divergence is asymmetric. In cases where $p(x)$ is close to zero, but $q(x)$ is significantly non-zero, the q's effect is disregarded. It could cause buggy results when we just want to measure the similarity between two equally important distributions.

Introducing the energy model

A typical Boltzmann machines employs a probability distribution

$$p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \frac{f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{Z(\boldsymbol{\Theta})},$$

where $f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})$ is given by a so-called energy model. If we assume that the random variables x_i and h_j take binary values only, for example $x_i, h_j = \{0, 1\}$, we have a so-called binary-binary model where

$$f(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = -E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \sum_{x_i \in \boldsymbol{X}} x_i a_i + \sum_{h_j \in \boldsymbol{H}} b_j h_j + \sum_{x_i \in \boldsymbol{X}, h_j \in \boldsymbol{H}} x_i w_{ij} h_j,$$

where the set of parameters are given by the biases and weights $\boldsymbol{\Theta} = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{W}\}$. **Note the vector notation** instead of x_i and h_j for f. The vectors $\boldsymbol{x}$ and $\boldsymbol{h}$ represent a specific instance of stochastic variables x_i and h_j. These arrangements of $\boldsymbol{x}$ and $\boldsymbol{h}$ lead to a specific energy configuration.

More compact notation

With the above definition we can write the probability as

$$p(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta}) = \frac{\exp\left(\boldsymbol{a}^T \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{h} + \boldsymbol{x}^T \boldsymbol{W} \boldsymbol{h}\right)}{Z(\boldsymbol{\Theta})},$$

where the biases $\boldsymbol{a}$ and $\boldsymbol{h}$ and the weights defined by the matrix $\boldsymbol{W}$ are the parameters we need to optimize.

Derivatives

Since the binary-binary energy model is linear in the parameters a_i, b_j and w_{ij}, it is easy to see that the derivatives with respect to the various optimization parameters yield expressions used in the evaluation of gradients like

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial w_{ij}} = -x_i h_j,$$

and

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial a_i} = -x_i,$$

and

$$\frac{\partial E(\boldsymbol{x}, \boldsymbol{h}; \boldsymbol{\Theta})}{\partial b_j} = -h_j.$$

More details on Boltzmann machines

For a binary-binary model, these are the equations for the various gradients which are needed in the setup of a neural network. For more details see https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week11/ipynb/week11.ipynb

Code example

Here we provide a simple Python code for a classical binary-binary Boltzmann applied to the MNIST data set.

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load and preprocess MNIST
def load_binarized_mnist():
    print("Downloading MNIST...")
    mnist = fetch_openml('mnist_784', version=1)
    X = mnist.data.astype(np.float32) / 255.0
    X = (X > 0.5).astype(np.float32)  # Binarize
    return X

class RBM:
    def __init__(self, n_visible, n_hidden, learning_rate=0.1):
        self.n_visible = n_visible
        self.n_hidden = n_hidden
        self.learning_rate = learning_rate

        # Initialize weights and biases
        self.W = np.random.normal(0, 0.01, size=(n_visible, n_hidden))
        self.v_bias = np.zeros(n_visible)
        self.h_bias = np.zeros(n_hidden)

    def sigmoid(self, x):
```

Quantum Boltzmann Machines (QBMs)

A Quantum Boltzmann Machine (QBM) extends a classical BM by replacing each binary unit with a qubit and generalizing the energy to a quantum Hamiltonian.
Instead of a classical energy, one defines a Hamiltonian $H(\mathbf{\Theta})$ whose parameters $\mathbf{\Theta}$ (biases and couplings) play the role of the RBM weights. The model's density operator is the state (these are elements from quantum statistical mechanics)

$$\rho(\boldsymbol{\theta}) = \frac{\exp -(\beta H(\mathbf{\Theta}))}{Z(\mathbf{\Theta})},$$

with inverse temperature β (set to 1 as we did for the classical Boltzmann machine) and partition function $Z = \mathrm{Tr}(\exp -\beta H)$.
The probability of observing a visible configuration v is obtained by measuring ρ in the computational basis (and tracing out hidden qubits if any). In principle, the quantum model should be able to include more correlations via superposition and entanglement.

Observables

Observables are expectation values

$$\langle \hat{O} \rangle = \mathrm{Tr}(\rho \hat{O}).$$

In the QBM context, one is interested in the probability $p(v)$ of measuring the visible qubits in computational basis state v. If the full thermal state lives on both visible and hidden qubits, this probability is

$$p_\Theta(v) = \mathrm{Tr}\left[\Pi_v^{(\mathrm{vis})} \rho(\Theta)\right],$$

where $\Pi_v^{(\mathrm{vis})} = |v\rangle\langle v|$ acts on the visible subspace.
Equivalently, one may *trace out* the hidden qubits and work with the reduced density matrix on the visible subsystem. Computing these probabilities requires preparing or approximating the Gibbs state of H. In practice this is done either by quantum simulators, quantum annealers, or variational algorithms (add refs here).

Quantum Boltzmann Machines

The model distribution over classical bitstrings v is given by the diagonal of the quantum Gibbs (see whiteboard notes for definition of quantum Gibbs state) state $\rho = e^{-H}/Z$.
A straightforward choice is a stochastic Hamiltonian that is diagonal in the computational basis (in our case given in terms of only the Pauli-Z operator), which yields a probability distribution very similar to a classical BM. More generally one can allow non-commuting terms (e.g. Pauli-X fields) to introduce quantum correlations . In fact, Amin et al. (2018) introduced a QBM where the training is done by bounding the quantum probabilities and sampling from the transverse-field Ising Hamiltonian . However, non-commutativity makes exact training harder, so many proposals use either special Hamiltonians or variational approximations.

Restricted QBM (RQBM)

A Restricted Quantum Boltzmann Machine (RQBM) (also called Quantum RBM or QRBM) enforces a bipartite structure analogous to the classical RBM: no hidden-hidden interactions, and possibly limited hidden-visible connectivity. The simplest RQBM Hamiltonian can be written (up to local Pauli bases) as

$$H(\mathrm{a}, \mathrm{b}, W, V) = \sum_{i=1}^{n_v} a_i Z_i + \sum_{j=1}^{n_h} b_j Z_j + \sum_{i,j} w_{ij}\, Z_i Z_j \; + \sum_{i<i'} V_{ii'}\, Z_i Z_{i'} \,.$$

Here Z_i and Z_j are Pauli-Z operators on the visible and hidden qubits respectively, a_i, b_j are biases, w_{ij} are visible-hidden couplings, and $V_{ii'}$ are possible visible-visible couplings. (Classically, $V = 0$ in an RBM). Importantly, there are no hidden-hidden ZZ terms in this restricted model. The above equation is a direct quantum analogue of the RBM energy function, promoting it to an operator acting on qubits. .

Energy-Based Training Objective and Gradients

RQBM training is analogous to the classical case: we have a dataset of bitstrings $\{v^{(k)}\}$ from an unknown distribution $p_{\mathrm{data}}(v)$.

The goal is to adjust the Hamiltonian parameters Θ so that the model distribution

$$p_\Theta(v) = \langle v|\rho(\Theta)|v\rangle,$$

approximates $p_{\mathrm{data}}(v)$. Equivalently, one can view the data distribution as a target density matrix η (diagonal in the computational basis) and minimize the quantum relative entropy (quantum KL divergence)

$$S(\eta|\rho(\Theta)) = \mathrm{Tr}\big[\eta \ln \eta\big] - \mathrm{Tr}\big[\eta \ln \rho(\Theta)\big].$$

Non-negative loss

This loss is non-negative and equals zero only when $\eta = \rho(\theta)$. Writing $\rho = e^{-H}/Z$, one finds the gradient of the relative entropy (for parameter Θ in H) as

$$\frac{\partial}{\partial \Theta} S(\eta|\rho) = \mathrm{Tr}\Big[\eta\, \partial_\Theta(\beta H + \ln Z)\Big] = \beta\Big(\mathrm{Tr}[\eta\, \partial_\theta H] - \mathrm{Tr}[\rho\, \partial_\Theta H]\Big).$$

In other words,

$$\nabla_\Theta S = \beta\Big(\langle \partial_\Theta H\rangle_{\mathrm{data}} - \langle \partial_\Theta H\rangle_{\mathrm{model}}\Big).$$

Analogy with classical RBM

This is directly analogous to the classical RBM gradient: the update for each parameter is proportional to the difference between its expectation under the data distribution and under the model's Gibbs distribution

$$\eta|\rho) = \mathrm{Tr}[\eta \ln \eta] - \mathrm{Tr}[\eta \ln \rho].$$

In practice, one computes $\langle \partial_\Theta H \rangle_{\mathrm{data}}$ by averaging over the training set, and estimates $\langle \partial_\theta H \rangle_{\mathrm{model}}$ by sampling from the quantum model.

Gibbs sampling

Note that preparing exact Gibbs samples of a non-commuting Hamiltonian is hard. Many methods have been proposed to approximate the model expectation. For example, one may use a bound on the quantum free energy, or perform contrastive divergence with a quantum device. Recent theoretical work shows that minimizing the relative entropy in QBM training can be done with stochastic gradient descent in polynomial sample complexity under reasonable assumptions .

Parameter Optimization and Variational Techniques

Given the gradient above, one can optimize Θ by standard gradient-based methods (SGD, Adam, etc.). In a gate-based setting, we implement the RQBM Hamiltonian via a parameterized quantum circuit (ansatz) and use variational quantum algorithms (VQAs). Each parameter in H is encoded as a gate angle or circuit parameter. The gradient of a circuit expectation can be obtained by the parameter-shift rule or automatic differentiation.

Variational Quantum Boltzmann machines (VQBM)

See whiteboard notes.

Implementation with PennyLane

Below we sketch a toy example: a two-qubit Quantum Circuit Born Machine (QCBM), which generates a probability distribution via a parameterized quantum circuit. While a QCBM is not exactly a QBM (it has no thermal state), it serves to show how to code a quantum generative model.

```
import pennylane as qml
from pennylane import numpy as np

# Use a 2-qubit simulator
dev = qml.device('default.qubit', wires=2)

# Define a simple 2-qubit QCBM circuit
@qml.qnode(dev)
def circuit(params):
    # params = 4 angles
    qml.RX(params[0], wires=0)
    qml.RY(params[1], wires=1)
    qml.CNOT(wires=[0, 1])
    qml.RX(params[2], wires=0)
    qml.RY(params[3], wires=1)
    return qml.probs(wires=[0,1])  # return probabilities of |00>,|01>

# Initialize random parameters
params = np.random.randn(4, requires_grad=True)

# Get the output distribution from the circuit
```

Training of a QBM

In practice, training a QBM in PennyLane would involve preparing quantum states corresponding to the Gibbs distribution. One could use functions like qml.qaoa.cost.Hamiltonian or custom circuits to approximate $\exp -\beta H$, and then use PennyLane's optimizers. Also, PennyLane can interface with PyTorch or TensorFlow, enabling hybrid optimization of quantum circuits with classical parameters (e.g. the Hamiltonian weights).

More code examples

This code defines a target distribution (e.g., classical binary data) using a Variational Quantum Boltzmann machines (VQBM). At each epoch it trains the model and samples the model and final computes the histogram probabilities.

```
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from collections import Counter

# Config
num_visible = 2
num_hidden = 2
num_qubits = num_visible + num_hidden
epochs = 50
shots = 1000

dev = qml.device("default.qubit", wires=num_qubits, shots=shots)

# Target data (biased toward '11' and '00')
target_bitstrings = ['11', '11', '11', '00', '00', '01']
target_counts = Counter(target_bitstrings)
target_probs = {
    format(i, f'0{num_visible}b'): target_counts.get(format(i, f'0{num
    for i in range(2**num_visible)
}
```

Summary of steps in PennyLane implementation

1. Define a quantum device (qml.device) with enough wires for visibles+hidden.
2. Construct a parametric quantum circuit (ansatz) that depends on trainable parameters (e.g. angles in rotations and entangling gates).
3. If modeling an RQBM, one might clamp visible qubits (preparing them according to training data) and apply gates only to hidden qubits.
4. Measure relevant observables: one can return probabilities (probs) or expectation values (expval) of Pauli operators, depending on the chosen loss function.
5. Define a cost function, such as the negative log-likelihood or a distance between output and target distribution.

Use an optimizer (e.g. gradient descent, Adam) with PennyLane's gradient calculations to update parameters.